Analyse Statique - Caslog

PITOU Bruny 97002285 - RIVIERE Gérard 98000567

dirigé par :

Frédéric Mesnard et Etienne Payet

Université de la Réunion

27 mai 2003

Remerciements.

Nous souhaitons remercier toutes les personnes qui nous ont permis de mener à bien la réalisation de ce TER. En particulier Frédéric Mesnard et Etienne Payet, responsables de TER, pour leurs précieux conseils dans l'étude théorique des complexités utilisé par Caslog.

Nous tenons également à remercier Roberto Bagnara ainsi que toute son équipe de l'université de Parme, pour leurs aides dans la conexion de PURRS à Caslog.

Table des matières

1 Présentation de notre TER **3**

2 L'analyse statique des prédicats **4**
2.1 Rappels et définitions 4
2.2 La taille d'un terme 5
2.3 Comment définir un programme pour Caslog ? 6
2.4 Un exemple d'utilisation de Caslog 6
2.4.1 Le graphe de dépendance 7
2.4.2 La complexité en espace 7
2.4.3 La taille des relations et le nombre de solutions en fonction de la taille des arguments en entrée : 9
2.4.4 La taille de la relation select : 12
2.4.5 Calcul du nombre de solutions de select : 13
2.4.6 La complexité en temps : 14

3 Exemples **15**
3.1 perm 15
3.1.1 La taille des sorties en fonction des entrées : 15
3.1.2 La taille de la relation 16
3.1.3 Le nombre de solutions : 16
3.1.4 La complexité en temps 17
3.2 permSort 18
3.2.1 Calcul de complexité pour *sorted* 18
3.2.2 Calcul de complexité pour permSorted 19
3.3 append 21
3.3.1 Le graphe de dépendance des littéraux 21
3.3.2 La taille des arguments de sortie en fonction des entrées 21
3.3.3 La taille de la relation 22
3.3.4 Le nombre de solutions : 22
3.3.5 Complexité en temps 22
3.4 quickSort 23
3.4.1 Le graphe de dépendance des littéraux 23
3.4.2 Calcul de complexité pour le prédicat *part* 24
3.4.3 Calcul de complexité pour le prédicat *qsort* 26

4 Programmation de Caslog en ISO-Prolog **28**
4.1 Le fonctionnement de Caslog 28
4.2 Les différents modules de Caslog 29
4.3 Les modifications apportées 32
4.3.1 Les modifications pour faire fonctionner sous une version récente de SICStus 32

4.3.2 Les modifications pour faire fonctionner sous GNU-Prolog . . . 33
4.3.3 Mise en ISO-Prolog . . . 33

5 Connexion de PURRS à Caslog **35**
5.1 Présentation de PURRS . . . 35
5.1.1 Les principales fonctionnalités . . . 35
5.1.2 Format des équations d'entrées . . . 35
5.1.3 Les symboles acceptés pour ces équations . . . 36
5.2 Méthodes utilisées pour la connexion . . . 36
5.2.1 Connexion effectuée par un shell . . . 36
5.2.2 Connexion effectuée directement par Prolog . . . 37
5.2.3 Exemple de problèmes rencontrés et résolus . . . 37
5.3 Différence entre Caslog et PURRS . . . 41

6 Méthode de travail **42**

7 Conclusion **43**

Chapitre 1

Présentation de notre TER

Nous avons voulu mettre en avant les connaissances acquises en Licence et en programmation logique avec Prolog et Mercury.

Notre préference pour la programmation logique et l'étude des complexités nous a poussé à choisir ce TER.

Caslog signifie Cost Analysis of Logics Programs écrit par Saumya K. Debray et Nai-Wei Lin au département informatique de l'Université d'Arizona.

Tout d'abord, nous allons étudier de manière théorique le fonctionnement de Caslog et ses différentes possibilités de calcul de complexité pour tout programme logique défini :

- **la complexité en espace ;**
- **la taille d'une relation i.e. le nombre de tuples dans son graphe ;**
- **le nombre de solutions en fonction de la taille des arguments en entrée ;**
- **la complexité en temps en fonction de la taille des arguments en entrée.**

On s'interesse aux complexités dans le pire des cas.

Nous avons tout d'abord mis Caslog au format ISO-Prolog et pour commencer, nous avons utilisé GNU-Prolog (conforme à la norme ISO-Prolog).

Ensuite, on a fait la connexion d'un résolveur d'équations récurrentes (PURRS) avec Caslog. Ce résolveur a été implémenté par l'équipe de Roberto Bagnara à l'Université de Parme.

Chapitre 2

L'analyse statique des prédicats

2.1 Rappels et définitions

- Un **programme** est composé d'une ou plusieurs **clauses**.
- Une **clause** est une disjonction d'un nombre fini de **littéraux**. En programmation logique, les **clauses sont de Horn** : par définition, elles possèdent exactement un littéral positif et sont de la forme :
$$\forall(p \vee \neg q_1 \vee \neg q_2 \vee ... \vee \neg q_n)$$
noté en prolog :
$$p :- q_1, q_2, ..., q_n.$$
- Un **littéral** est soit un atome soit la négation d'un atome.
- Un atome est de la forme $p(t_1, t_2, ..., t_n)$ où p est un symbole de prédicat n-aire et $t_1, .., t_n$ sont des termes.
- Un **terme** est soit une variable, soit un objet de la forme $f(t_1, .., t_n)$ où f est un symbole de fonction n-aire et $t_1, ..., t_n$ sont des termes.
- Un terme clos est un terme qui ne contient pas de variable.

Maintenant, on va définir le vocabulaire qui sera utilisé dans ce rapport :
- **head** : la tête de la clause ;
- **body** : le corps de la clause ;
- pour simplifier la compréhension :
  - tous les termes en entrée dans un programme seront appelés arguments d'entrée ou **input** ;
  - tous les termes en sortie dans un programme seront appelés arguments de sortie ou **output** ;
- $head[i]$ avec $i \in \mathcal{N}$: la taille du $i^{ème}$ argument dans la tête ;
- $body_j[i]$ avec $i \in \mathcal{N}$: la taille du $i^{ème}$ argument du $j^{ème}$ littéral du corps de la clause ;
- deux clauses d'un prédicat sont **mutuellement exclusives** si, en fonction des termes en entrée de l'atome $p(t_1, t_2, ..., t_n)$, au plus une des deux têtes de clause s'unifie avec $p(t_1, t_2, ..., t_n)$. Par exemple, considérons la procédure logique :
$$list([]).$$
$$list([_|Xs]) :- list(Xs).$$
avec :
$$:- mode(list/1, [+]).$$
Les deux clauses définissant la procédure sont mutuellement exclusives.

En revanche avec :

$$:- mode(list/1, [-]).$$

ce n'est pas le cas.

2.2 La taille d'un terme

On note :

$$taille_m : \mathcal{H} \longrightarrow \mathcal{N}_\perp$$

$\mathcal{H}$ est l'univers de Herbrand(l'ensemble des termes clos).

$\mathcal{N}_\perp$ est l'ensemble des entiers naturels auquel on ajoute un symbole spécial $\perp$ signifiant "indéterminé".
Soit $t \in \mathcal{H}$, la valeur m donne le type de l'entrée que l'on utilisera pour calculer la taille :

1. si m est integer_value, on calcule la taille suivant la valeur des entiers :
   - $taille_{integer_value}(t) = n$ si t est un entier n ;
   - $taille_{integer_value}(t) = \odot(taille_{integer_value}(t_1), ..., taille_{integer_value}(t_n))$ si $t = \odot(t_1, ..., t_n)$ pour toute fonction arithmétique $\odot$;
   - $taille_{integer_value}(t) = \perp$ sinon.
2. Si m est $list_length$, on calcule la taille suivant la longueur des listes :
   - $taille_{list_length}(t) = 0$ si t=[] ;
   - $taille_{list_length}(t) = 1 + taille_{list_length}(t_1)$ si $t = [_|t_1]$;
   - $taille_{list_length}(t) = \perp$ sinon.
3. Si m est $term_depth$, on calcule la taille suivant la profondeur des termes, c'est-à-dire si t est un arbre, la taille de t sera la profondeur maximale de l'arbre :
   - $taille_{term_depth}(t) = 0$ si t est une constante ;
   - $taille_{term_depth}(t) = 1 + max\{taille_{term_depth}(t_i) | 1 \leq i \leq n\}$ si $t = f(t_1, ...t_n)$;
   - $taille_{term_depth}(t) = \perp$ sinon.
4. Si m est $term_size$, on calcule la taille suivant la taille des termes, c'est-à-dire si on considère t comme un arbre, la taille de t sera le nombre de noeuds de l'arbre :
   - $taille_{term_size}(t) = 1$ si t est une constante ;
   - $taille_{term_size}(t) = 1 + \sum_{i=1}^{n} \{taille_{term_size}(t_i)\}$ si $t = f(t_1, ...t_n)$;
   - $taille_{term_size}(t) = \perp$ sinon.

2.3 Comment définir un programme pour Caslog ?

Exemple :

$: -mode(p/3, [+, +, -]).$
$: -measure(p/3, [int, int, int]).$
$: -domain(p/3, [[1-3], [1-3], [1-3]]$
$p(X, Y, Z) : -q(X, Y), r(Y, Z).$

---

1. Le **mode** est **obligatoire** pour définir les entrées et les sorties.
   - $[+, +, -]$ avec $+$: input et $-$: ouput (deux arguments en entrée et un en sortie) ;
   - $p/3$: p est le nom du prédicat que l'on a défini d'arité 3.
2. La **mesure** est **obligatoire** $[int, int, int]$, c'est une liste de noms de mesures dans l'ordre des arguments du prédicat, c'est-à-dire pour chaque terme du prédicat p, on donne la mesure et le type de l'entrée que l'on utilisera pour calculer les complexités.
   Dans cet exemple, la mesure pour chaque terme est "integer_value".
   Donc, le calcul des complexités va se faire à partir des valeurs de chaque terme qui sont supposés être des entiers.
   Il existe plusieurs types de mesure :
   - void : les calculs de complexité ne dépendent pas de la taille du terme associé (la taille de ce terme vaut 0) ;
   - int : integer_value ;
   - length : list_length ;
   - size : term_size ;
   - depth : term_depth.
3. Le **domaine** n'est pas obligatoire. [[1-3],[1-3],[1-3]] est une liste de domaines de définitions de chaque terme.

4. La définition du prédicat, composé d'une tête (le **head** : un littéral positif) et d'un corps de littéraux (**body**) :

$$p(X, Z) : -q(X, Y), r(Y, Z).$$

2.4 Un exemple d'utilisation de Caslog

$: -mode(select/3, [-, +, -]).$
$: -measure(select/3, [size, length, length]).$
$select(X, [X|Xs], Xs).$
$select(X, [Y|Ys], [Y|Zs]) : -select(X, Ys, Zs).$

Dans cet exemple, nous définissons :

- **le mode** :
  on a un prédicat *select* d'arité 3 : $select/3$,
  avec le deuxième argument en entrée, le premier et le troisième en sortie ;

- **la mesure** :
  le prédicat d'arité 3 : $select/3$, puis la liste des types des arguments ;
- **la définition du prédicat** *select* :
  la première clause représente la condition d'arrêt et son corps est vide, la deuxième clause est composée de :
  - un membre gauche $select(X, [Y|Ys], [Y|Zs])$: **head** ;
  - un membre droit $select(X, Ys, Zs)$: **body**.

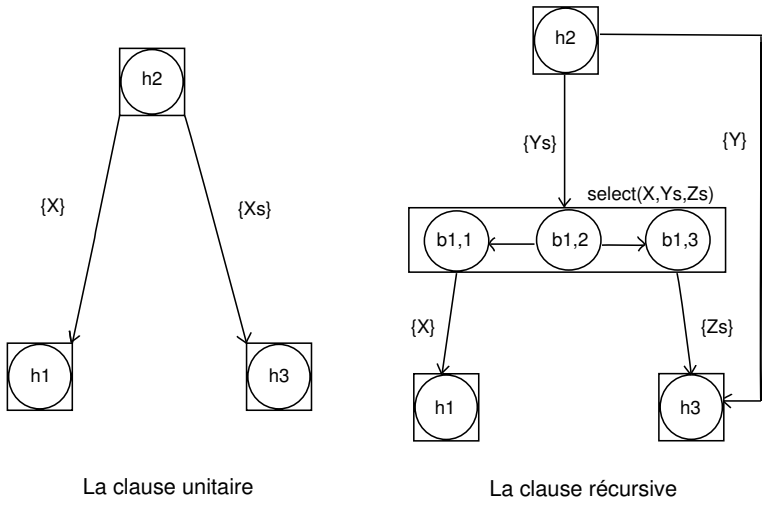
2.4.1 Le graphe de dépendance

Le graphe de dépendance des littéraux est un graphe orienté acyclique qui représente les dépendances entre les littéraux.

Chaque noeud représente un littéral, qui est lui-même composé d'un ensemble de noeuds qui correspondent à la position des arguments dans le littéral.

Il existe un arc entre deux noeuds (étant des arguments b1,2 et b2,1), si les variables apparaissant dans b1,2 sont utilisées pour construire b2,1.

En reprenant l'exemple précédent, on a les graphes suivants :



2.4.2 La complexité en espace

On va maintenant calculer la taille des arguments de sortie en fonction des arguments en entrée à l'aide de :

$$Sz_p^b : \mathcal{H} \longrightarrow \mathcal{N}_{\perp,\infty}$$

$\mathcal{H}$: l'Univers de Herbrand (l'ensemble des termes clos)

$\mathcal{N}_{\perp,\infty}$: on aura en sortie un entier naturel ou $\perp$ (indéterminé) ou ∞ (infini)

b : la position d'un argument de sortie
dans cet exemple, b peut être égal à 1 ou à 3

p : le nom du prédicat
Autre notation :

$head[i]$: la taille du $i^{ème}$ argument du prédicat situé dans la tête de la clause.

$body_i[j]$: la taille du $j^{ème}$ argument, du $i^{ème}$ prédicat situé dans le corps de la clause.

Pour *select*, on a la complexité en espace :

- **Calcul de head[1] :**

  Pour la seconde clause, on a :

  $head[1] = taille(X)$

  et d'après le graphe de dépendance ci-dessus,

  $head[1] \leq body_1[1]$

  (car $head[1]$ est obtenu à partir de $body_1[1]$, notés h1 et b1,1 dans le graphe)
  or :

  $body_1[1] \leq Sz^{(1)}_{select}(body_1[2])$

  car $body_1[1]$ dépend de $body_1[2]$ d'après le graphe.
  Et $body_1[2]$ est obtenu à partir de $head[2] - taille(Y)$ ($taille(Y) = 1$ car Y est de mesure *size*),

  d'où :

  $head[1] \leq Sz^{(1)}_{select}(head[2] - 1)$.

  Pour la première clause, d'après le graphe de dépendance :
  $head[1] = head[2] - diff([X|Xs], X)$
  car h1 est obtenu à partir de h2 privé de Xs,
  et $diff([X|Xs], X) = diff(Xs, X) + 1$

  or on ne connaît pas la taille de Xs,
  d'où :
  $diff([X|Xs], X) = \bot$
  donc :
  $head[1] = \bot$

  on a :
  $Sz^{(1)}_{select}(x) = Sz^{(1)}_{select}(x - 1)$
  $Sz^{(1)}_{select}(x) = \bot$

  Donc :

  $$\boxed{Sz^{(1)}_{select} \equiv \lambda x.\bot}$$

- **Calcul de head[3] :**

  D'après le graphe de dépendance, $head[3]$ dérive de $body_1[3]$
  et $taille([Y|Zs]) = taille(Y) + taille(Zs)$,
  or, $[Y|Zs]$ est de mesure *length* donc $taille(Y) = 1$
  d'où :
  $head[3] = taille[Zs] + 1$
  et toujours d'après le graphe de dépendance au niveau des arguments, la taille de Zs est fonction de $body_1[2]$
  d'où :
  $head[3] \leq Sz^{(3)}_{select}(body_1[2]) + 1$

  et $body_1[2] = head[2] - 1$

donc :

$$head[3] \leq Sz^{(3)}_{select}(head[2]-1)+1$$

Pour la première clause :

$$head[3] \leq head[2]-1$$

on obtient une équation différentielle :

$$Sz^{(3)}_{select}(x) = Sz^{(3)}_{select}(x-1)+1$$
$$Sz^{(3)}_{select}(x) = x-1$$

Donc :

$$\boxed{Sz^{(3)}_{select} \equiv \lambda x.x-1}$$

2.4.3 La taille des relations et le nombre de solutions en fonction de la taille des arguments en entrée :

D'après l'article *Cost Analysis of logic Programs* de Saumya K. Debray et Nai-Wei Lin, on a les théorèmes suivants.

On note $\mathcal{B}_{\{T\}}$ le nombre d'instanciations pour un tuple de termes T dans une clause. Si T contient un ensemble de variables $\{X_1, ..., X_m\}$, et soit X_i appartenant à cet ensemble, donc $N_{\{X_i\}}$ est le nombre d'instanciations de la variable X_i.

Théorème 1 *Si une variable X a n occurrences dans le corps d'une clause et si les nombres d'instanciations possibles pour ces différentes occurrences sont $k_1, .., k_n$ alors $\mathcal{B}_{\{X\}} \leq min\{k_1, ..., k_n\}$.*

ex : $p(Y) : -q(Y), r(Y)$
$q(0).\ q(1).$
$r(0).\ r(1).\ r(2).$

Le nombre de d'instanciations de Y est 2.

Théorème 2 *Soit T un (tuple de) terme(s) dans une clause. Si T contient un ensemble de variables $S = \{X_1, ..., X_m\}$, tel que $\mathcal{B}_{\{X_i\}} \leq N_{\{X_i\}}$, avec $\mathcal{B}_{\{X_i\}}$ le nombre d'instanciations de la variable X_i,*
pour $1 \leq i \leq m$, alors $\mathcal{B}_{\{T\}} \leq \prod_{i=1}^{m} N_{\{X_i\}}$, avec $\mathcal{B}_{\phi} = 1$ (le nombre d'instanciation d'un tuple vide).

ex : Si un terme $f(X, Y, X)$ avec X ayant deux instances a et b et Y deux instances c et d alors le nombre total d'instanciations possibles est 4, c'est-à-dire $f(a, c, a)$, $f(b, c, d)$, $f(a, d, a)$, $f(b, d, b)$.

Lemme :
Si le nombre d'instanciations de chaque occurrence de variable dans un tuple de termes T est déterminé, alors nous pouvons définir une fonction qui calcule le nombre d'instanciations pour un tuple de termes.

Soit T contenant un ensemble de variables, $S = \{X_1, ..., X_m\}$ et $N_{\{X_i\}}$ le nombre d'instanciations de X_i.
Nous avons :

$$instance(T) = \prod_{i=1}^{m} N_{\{X_i\}}.$$

Nous associons ainsi pour chaque prédicat un "binding pattern" de la forme $< Rel_p, Sol_p >$ avec Rel_p la taille de la relation d'un prédicat p et Sol_p le nombre de solutions de p.

Soit la clause :

$$p(x_0, y_0) : -q_1(x_1, y_1), ..., q_n(x_n, y_n)$$

- x_i : argument en entrée ;
- y_i : argument en sortie ;
- $vars(t)$: l'ensemble des variables du tuple t (éléments non clos) ;
- $lits(v)$: l'ensemble des littéraux où la variable v apparaît ;
- n_i : la taille des entrées du littéral q_i.

Donc d'après le théorème 1, on a :

$$\mathcal{B}_{\{v\}} \leq N_{\{v\}} = min\{Rel_j | j \in lits(v)\}.$$

Et d'après le théorème 2, pour toutes **entrées** x_i d'un littéral q_i, on peut estimer son nombre d'instanciations par :

$$\mathcal{B}_{\{x_i\}} \leq I_i = min\{instance(x_i), Rel_{q_i}\},$$

pour toutes **sorties** y_i d'un littéral q_i on peut estimer son nombre d'instanciation par :

$$\mathcal{B}_{\{x_i\}} \leq O_i = min\{I_i \times Sol_{q_i}(n_i), Rel_{q_i}\}$$

avec n_i la taille des entrées du littéral q_i.

Donc, on aura pour toutes variables v appartenant à y_i :

$$\mathcal{B}_{\{v\}} \leq N_{\{v}\} = min\{O_i, Rel_j | j \in lits(v)\},$$

et pour toutes les variables de la clause, on aura :

$$\mathcal{B}_{\{x_0,y_0\}} \leq Rel_p = instance((x_0, y_0)).$$

Voici l'algorithme du "binding pattern" :

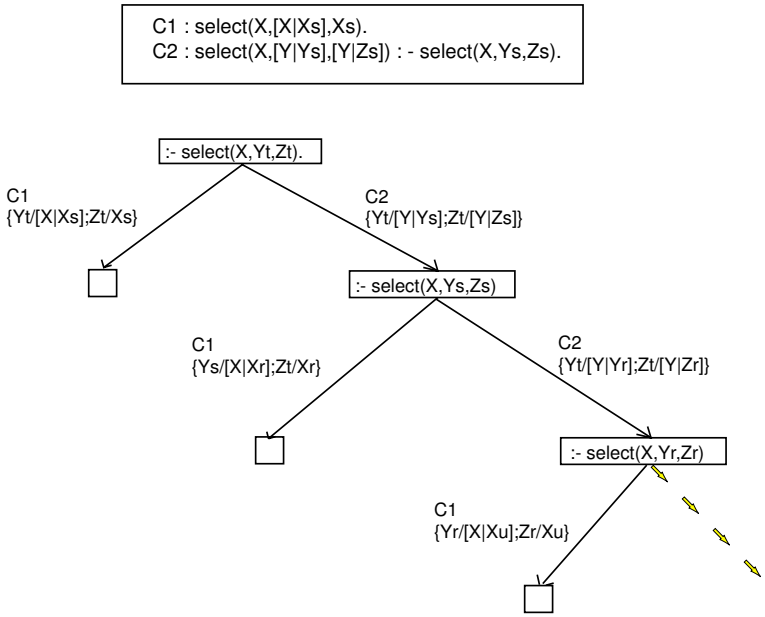
début
/*La taille de la relation p*/
si p est récursif **alors** $Rel_p := \infty$
sinon faire
pour chaque variable $v \in vars(x_0)$ **faire**
/** *d'après le théorème 1* **/
$N_{\{v\}} := min\{Rel_j | j \in lits(v)\}$;
fin pour
pour $i := 1$ **à** n **faire**
$I_i := min\{instance(x_i), Rel_{q_i}\}$;
$O_i := min\{I_i \times Sol_{q_i}(n_i), Rel_{q_i}\}$;
pour chaque variable $v \in vars(y_i)$ **faire**
$N_{\{v\}} := min\{O_i, Rel_j | j \in lits(v)\}$;
fin pour
fin pour
/** *d'après le lemme* **/
$Rel_p := instance((x_0, y_0))$;
fin pour

/*Le nombre de solutions de p*/
pour chaque variable $v \in vars(x_0)$ **faire**
$N_{\{v\}} := 1$;
fin pour
pour $i := 1$ **à** n **faire**
$I_i := min\{instance(x_i), Rel_{q_i}\}$;
$O_i := min\{I_i \times Sol_{q_i}(n_i), Rel_{q_i}\}$;
pour chaque variable $v \in vars(y_i)$ **faire**
$N_{\{v\}} := min\{O_i, Rel_j | j \in lits(v)\}$;
fin pour
fin pour
$Sol_p := instance((y_0))$;
fin pour

2.4.4 La taille de la relation select :

La taille d'une relation est le nombre de tuples dans son arbre de dérivation.

L'arbre de dérivation de select :



Comme **select** est récursive alors son arbre de dérivation va comporter un nombre infini de tuples (le nombre de feuilles), donc

$$\boxed{Rel_{select} = \infty}$$

2.4.5 Calcul du nombre de solutions de select :

Y et Ys sont des variables d'entrées, donc $N_{\{Y\}} = N\{Ys\} = 1$.

On n'a qu'un littéral :

$I_1 = min\{instance(Ys), Rel_{select}\} = 1$
$O_1 = min\{1 \times Sol_{select}(body_1[2]), Rel_{select}\}$
$O_1 = Sol_{select}(head[2] - 1)$

X argument de sortie :
$N_{\{X\}} = min\{O1, Rel_{select}\}$
$N_{\{X\}} = Sol_{select}(head[2] - 1)$

Z_s est un argument de sortie :
$N_{\{Z_s\}} = min\{O_1, Rel_{select}\}$
$N_{\{Z_s\}} = Sol_{select}(head[2] - 1)$

Donc
$Sol_{select} = instance((X, [Y|Zs]))$
$= min\{N_{\{X\}} \times N_{\{Y\}} \times N_{\{Zs\}}, O_1\}$
$= Sol_{select}(head[2] - 1)$

D'où
$Sol_{select}(x) = Sol_{select}(x - 1)$
pour la première clause :
$Sol_{select}(x) = 1.$

Les deux clauses ne sont pas mutuellement exclusives, c'est-à-dire à chaque noeud de l'arbre de dérivation, on peut effectuer une unification avec la première clause et aussi avec la seconde clause.

Donc, le nombre de solutions du prédicat est la somme de Sol_{select} obtenue avec la première et la deuxième clause.
D'où
$Sol_{select}(x) = Sol_{select}(x - 1) + 1$

donc :

$$\boxed{Sol_{select} \equiv \lambda x.x}$$

2.4.6 La complexité en temps :

La complexité en temps d'une clause peut être majorée par la complexité en temps de l'unification de la tête, plus la complexité en temps du body (littéraux de droite). A cause du "backtracking", le temps nécessaire pour l'exécution d'un littéral dépend du nombre de solutions des littéraux qui le précèdent.
Soit :

$$T_p : \mathcal{N}_{\perp,\infty}^n \longrightarrow \mathcal{N}_\infty$$

p un prédicat avec n entrées.
Soit une clause C de la forme :

$$H : -L_1, ..., L_m$$

On a la complexité en temps suivante :

$$T_C(n) \leq \tau + \sum_{i=1}^{m} (\prod_{j \prec i} Sol_{L_j}(n_j)) T_{L_i}(n_i)$$

n est la taille des entrées de C et n_i la taille des entrées de chaque littéral L_i.

τ est le temps nécessaire à la résolution de la tête H de la clause avec les littéraux déjà résolus.

Si on utilise comme unité de complexité en temps le nombre de résolutions (correspondant au nombre de noeuds dans l'arbre de dérivation), alors τ vaut 1.
Si on utilise le nombre d'unifications (correspondant au nombre d'arcs dans l'arbre de dérivation), alors τ vaudra l'arité de la tête de C.

$j \prec i$ signifie que le littéral L_j précède L_i dans le graphe des dépendances de C.

En appliquant ce théorème sur l'exemple, on a :

$T_{select}(head[2]) \leq \tau + T_{select}(head[2] - 1)$
$T_{select}(head[2]) \leq \tau$ (pour la première clause).

On prendra comme unité de complexité en temps le nombre de résolutions,
$\tau = 1$.
De même comme les clauses de *select* sont mutuellement exclusives :
alors
$T_{select}(head[2]) = T_{select}(head[2] - 1) + 2$
et
$T_{select}(0) = 0$
donc :

$$\boxed{T_{select} \equiv \lambda x.2 \times x}$$

Chapitre 3

Exemples

3.1 perm

$: -mode(perm/2, [+, -]).$
$: -measure(perm, [length, length]).$
$perm([], []).$
$perm(X, [R|Rs]) : -select(R, X, Y), perm(Y, Rs).$

$select(X, [X|Xs], Xs).$
$select(X, [Y|Ys], [Y|Zs]) : -select(X, Ys, Zs).$

3.1.1 La taille des sorties en fonction des entrées :

- **Calcul de head[2] :**

$$\begin{aligned} head[2] &= taille(Rs) + 1 \\ &\leq body_2[2] + 1 \\ &\leq Sz^{(2)}_{perm}(body_2[1]) + 1 \\ &\leq Sz^{(2)}_{perm}(body_1[3]) + 1 \\ &\leq Sz^{(2)}_{perm}(Sz^{(3)}_{select}(body_1[2])) + 1 \\ &\leq Sz^{(2)}_{perm}(Sz^{(3)}_{select}(head[1])) + 1 \end{aligned}$$

Or

$$Sz^{(3)}_{select} \equiv \lambda x.x - 1$$

alors :

$$head[2] \leq Sz^{(2)}_{perm}(head[1] - 1) + 1$$

donc :

$$Sz^{(2)}_{perm}(x) = Sz^{(2)}_{perm}(x - 1) + 1$$

Et avec la première clause, on obtient :

$$Sz^{(2)}_{perm}(0) = 0$$

D'où :

$$\boxed{Sz^{(2)}_{perm} \equiv \lambda x.x}$$

3.1.2 La taille de la relation

Le prédicat *perm* est récursif, donc d'après l'algorithme :

$$Rel_{perm} = \infty.$$

3.1.3 Le nombre de solutions :

X est une variable d'entrée, donc $N_{\{X\}} = 1$.

On a deux littéraux :

- pour *select* :

$I_1 = min\{instance(X), Rel_{select}\} = 1$
$O_1 = min\{1 \times Sol_{select}(body_1[2]), Rel_{select}\}$
$O_1 = Sol_{select}(head[1])$

R argument de sortie :
$N_{\{R\}} = min\{O1, Rel_{select}\}$
$N_{\{R\}} = Sol_{select}(head[1])$

Y est un argument de sortie :
$N_{\{Y\}} = N_{\{R\}} = Sol_{select}(head[1]) = head[1].$

- Pour perm :

$I_2 = min\{instance(Y), Rel_{perm}\} = N_{\{Y\}}$
$O_2 = min\{I_2 \times Sol_{perm}(body_2[1]), Rel_{perm}\}$
$O_2 = N_{\{Y\}} \times Sol_{perm}(body_1[3])$
$O_2 = head[1] \times Sol_{perm}(Sz^{(3)}_{select}(head[1]))$
$O_2 = head[1] \times Sol_{perm}(head[1] - 1)$

Rs argument de sortie :
$N_{\{Rs\}} = min\{O2, Rel_{perm}\}$
$N_{\{Rs\}} = head[1] \times Sol_{perm}(head[1] - 1).$

Donc :

$Sol_{perm} = instance(([R|Rs]))$
$= min\{N_{\{R\}} \times N_{\{Rs\}}, O_2\}$
$= head[1] \times Sol_{perm}(head[1] - 1).$

Et :

$Sol_{perm}(0) = 1$
$Sol_{perm}(X) = X \times Sol_{perm}(X - 1).$

D'où :

$$\boxed{Sol_{perm} \equiv \lambda x.x!}$$

3.1.4 La complexité en temps

On prendra comme unité de complexité en temps le nombre de résolutions,
$\tau = 1$
$T_{perm}(head[1]) \leq \tau + T_{select}(head[1]) +$
$(Sol_{select}(head[1]) \times T_{perm}(body_2[1]))$.

Or

$Sol_{select}(head[1]) = head[1]$

et

$body_2[1] = body_1[3] = Sz^{(3)}_{select}(head[1]) = head[1] - 1$.

Donc :

$T_{perm}(head[1]) \leq 1 + T_{select}(head[1]) +$
$(head[1] \times T_{perm}(head[1] - 1)$.

Or

$T_{select} \equiv \lambda x.2x$.

D'où :

$T_{perm}(head[1]) \leq 1 + 2 \times head[1] +$
$head[1] \times T_{perm}(head[1] - 1)$

et

$T_{perm}(0) \leq 1$.

On obtient une équation récurrente de la forme :
$f(x) = 1 + 2x + (x \times f(x-1))$
$f(0) = 1$

Donc :

$$\boxed{T_{perm} \equiv \lambda x. \sum_{i=1}^{x} (\frac{3x!}{i!}) + 3x! - 2}$$

3.2 permSort

$: -mode(permSort/2, [+, -]).$
$: -mode(sorted/1, [+]).$
$: -measure(permSort/2, [length, length]).$
$: -measure(sorted/1, [length]).$
$permSort(Xs, Ys) : -perm(Xs, Ys), sorted(Ys)$
$sorted([]).$
$sorted([X]).$
$sorted([X, Y|Zs]) : -X =< Y, sorted([Y|Zs]).$

3.2.1 Calcul de complexité pour *sorted*

La taille de la relation :

Le prédicat *sorted* est récursif, donc

$$\boxed{Rel_{sorted} = \infty}$$

Le nombre de solutions :

Comme les trois clauses sont mutuellement exclusives, on peut en déduire que :

$$\boxed{Sol_{sorted} \leq 1}$$

La complexité en temps :

On prend comme unité de complexité en temps le nombre de résolutions, on a $\tau = 1$.

$T_{sorted}(head[1]) \leq 1 + 1 + T_{sorted}(head[1] - 1)$
$T_{sorted}(1) \leq 1$
$T_{sorted}(0) \leq 1.$

Donc :

$$\boxed{T_{sorted} \equiv \lambda x.(2 \times x - 1)}$$

3.2.2 Calcul de complexité pour permSorted

La taille des sorties en fonction des entrées :

$$\begin{aligned} head[2] &\leq body_2[1] \\ &\leq Sz^{(2)}_{permSort}(body_1[1]) \end{aligned}$$

or $Sz^{(2)}_{permSort} \equiv \lambda x.x$
d'où :

$$\begin{aligned} head[2] &\leq body_1[1] \\ &\leq head[1] \end{aligned}$$

donc :

$$\boxed{Sz_{permSort} \equiv \lambda x.x}$$

La taille de la relation :

Le prédicat $permSort$ est récursif, donc

$$\boxed{Rel_{permSort} = \infty}$$

Le nombre de solutions :

Xs est une variable d'entrée, donc $N_{\{Xs\}} = 1$,
pour le premier littéral :

$I_1 = min\{instance((Xs]), Rel_{perm}\} = 1$ or $Rel_{perm} = \infty$
$O_1 = 1 \times Sol_{perm}(head[1])$

$$N_{\{Ys\}} = O_1 = 1 \times Sol_{perm}(head[1]) = head[1]!$$

car $Sol_{perm} \equiv \lambda x.x!$
pour le deuxième littéral :

$I_2 = head[1]!$
$O_2 = 1 \times head[1]!$

d'où :

$$Sol_{permSort} = instance((Ys)) = head[1]!$$

Donc :

$$\boxed{Sol_{permSort} \leq \lambda x.x!}$$

La complexité en temps :

On prend comme unité de complexité en temps le nombre de résolutions, on a $\tau = 1$.
$T_{permSort}(head[1]) \leq 1 + T_{perm}(head[1]) + Sol_{perm}(head[1]) \times T_{sorted}(Sz^{(2)}_{perm}(head[1]))$
or :

$T_{perm} \equiv \lambda x. \sum_{i=1}^{x}(3 \times x!/i!) + 3 \times x! - 2$
$T_{sorted} \equiv \lambda x.(2x - 1)$
$Sz^{(2)}_{perm} \equiv \lambda x.x$

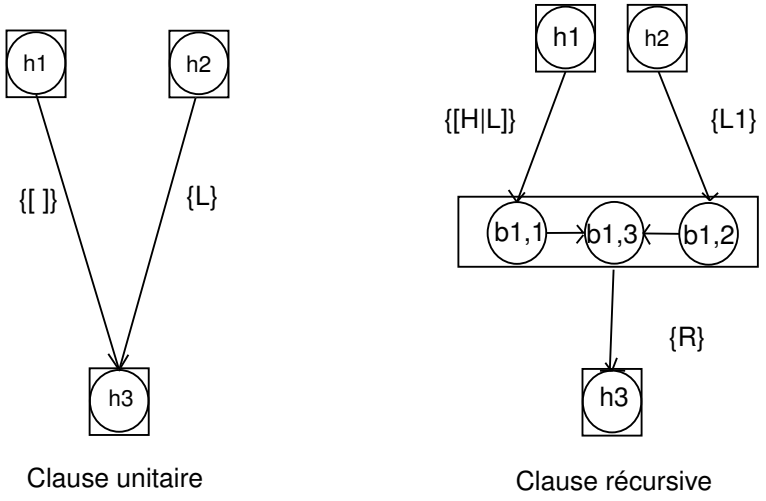
donc :

$$\boxed{T_{permSort} \equiv \lambda x.1 + (\sum_{i=1}^{x}(3 \times x!/i!) + 3 \times x! - 2) \times (2x - 1)}$$

3.3 append

$: -mode(append/3, [+, +, -])$
$: -measure(append/3, [length, length, length])$
$append([], L, L).$
$append([H|L], L_1, [H|R]) : -append(L, L_1, R).$

3.3.1 Le graphe de dépendance des littéraux



3.3.2 La taille des arguments de sortie en fonction des entrées

Calcul de head[3] :
pour la seconde clause, on a :
$head[3] = taille([H|R])$
$\leq body_1[3] + 1$
$\leq Sz^{(3)}_{append}(body_1[1], body_1[2]) + 1$
$\leq Sz^{(3)}_{append}(head[1] - 1, head[2]) + 1$
Et avec la première clause, on obtient :
$head[3] \leq head[2]$.
On a ainsi :

$$Sz^{(3)}_{append}(x, y) = Sz^{(3)}_{append}(x - 1, y) + 1$$

et

$$Sz^{(3)}_{append}(0, y) = y.$$

D'où :

$$f(x, y) = f(x - 1, y) + 1 = f(x - x, y) + x = y + x$$

Donc :

$$\boxed{Sz^{(3)}_{append} \equiv \lambda < x, y > .y + x}$$

3.3.3 La taille de la relation

Le prédicat *append* est récursif, donc d'après l'algorithme :

$$\boxed{Rel_{append} = \infty}$$

3.3.4 Le nombre de solutions :

H, L, L_1 sont des variables d'entrée,
donc :

$$N_{\{H\}} = N_{\{L\}} = N_{\{L_1\}} = 1$$

On a un littéral à droite :
$I_1 = min\{instance(L, L1), Rel_{append}\} = 1$
$O_1 = min\{1 \times Sol_{append}(body_1[1], body_1[2]), Rel_{append}\}$
$O_1 = Sol_{append}(head[1] - 1, head[2])$

comme R est un argument de sortie :
$N_{\{R\}} = min\{O_1, Rel_{append}\}$.
Donc :
$Sol_{append}(head[1], head[2)] = instance(([H|R]))$
$= min\{N_{\{H\}} \times N_{\{R\}}, O_1\}$.

Et :
$Sol_{append}(0, y) = 1$
$Sol_{append}(x, y) = Sol_{append}(x - 1, y)$.

D'où :

$$\boxed{Sol_{append} \equiv \lambda < x, y > .1}$$

3.3.5 Complexité en temps

On prend comme unité de complexité en temps le nombre de résolutions, on a $\tau = 1$.
$T_{append}(head[1], head[2]) \leq \tau + T_{append}(head[1] - 1, head[2])$.

Or
$T_{append}(0, head[2]) \leq 1$.

D'où :
$T_{append}(x, y) = 1 + T_{append}(x - 1, y))$
et
$T_{append}(0, y) = 1$

$T_{append}(x, y) = x + T_{append}(x - x, y) = x + 1$.
Donc :

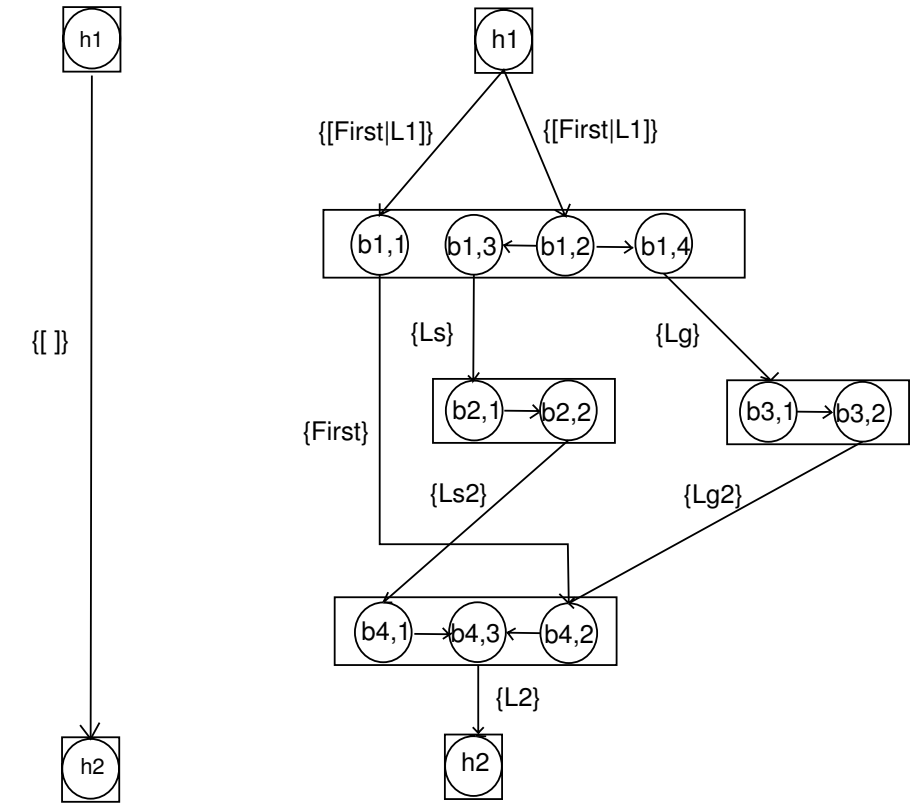
$$\boxed{T_{append} \equiv \lambda < x, y > .x + 1}$$

3.4 quickSort

$: -mode(qsort/2, [+, -]).$
$: -measure(qsort/2, [length, length]).$
$qsort([\,], [\,]).$
$qsort([First|L1], L2) : -part(First, L1, Ls, Lg),$
$\qquad\qquad\qquad\qquad\quad qsort(Ls, Ls2), qsort(Lg, Lg2),$
$\qquad\qquad\qquad\qquad\quad append(Ls2, [First|Lg2], L2).$

$: -mode(part/4, [+, +, -, -].$
$: -measure(part/4, [void, length, length, length]).$
$part(F, [], [], []).$
$part(F, [X|Y], [X|Y1], Y2) : -X \leq F, part(F, Y, Y1, Y2).$
$part(F, [X|Y], Y1, [X|Y2]) : -X > F, part(F, Y, Y1, Y2).$

3.4.1 Le graphe de dépendance des littéraux



3.4.2 Calcul de complexité pour le prédicat *part*

La taille des sorties en fonction des entrées :

- Pour la première clause :
  $Sz^{(3)}_{part}(0) = 0\,;\; Sz^{(4)}_{part}(0) = 0.$

- Pour la deuxième clause :
  $Sz^{(3)}_{part}(head[2]) = body_2[3] + 1$
  $Sz^{(3)}_{part}(head[2]) = Sz^{(3)}_{part}(head[2] - 1) + 1$
  et
  $Sz^{(4)}_{part}(head[2]) = Sz^{(4)}_{part}(head[2] - 1).$

- Pour la troisième clause :
  $Sz^{(3)}_{part}(head[2]) = Sz^{(3)}_{part}(head[2] - 1)$
  et
  $Sz^{(4)}_{part}(head[2]) = Sz^{(4)}_{part}(head[2] - 1) + 1.$

Donc :

$$\boxed{\begin{array}{l} Sz^{(3)}_{part} \equiv \lambda x.x \\ Sz^{(4)}_{part} \equiv \lambda x.x \end{array}}$$

La taille de la relation :

part est récursif, donc :

$$\boxed{Rel_{part} = \infty}$$

Le nombre de solutions :

F, X, Y sont des entrées, donc
$N_{\{F\}} = N_{\{X\}} = N_{\{Y\}} = 1$

- Pour la première clause :
  $Sol_{part}(0) = 1$

- Pour la deuxième clause :
  Le premier littéral dans le body n'a pas de variable de sortie.
  Le deuxième littéral :
  $I_2 = min\{instance(F, Y), Rel_{part}\}$
  $= N_{\{F\}} \times N\{Y\}$
  $O_2 = min\{I2 \times Sol_{part}(head[2] - 1), Rel_{part}\}$
  $= Sol_{part}(head[2] - 1)$
  $N_{\{Y1\}} = N_{\{Y2\}} = O2 = Sol_{part}(head[2] - 1)$

  $Sol_{part}(head[2]) = Sol_{part}(head[2] - 1)$

– Pour la troisième clause :

$$I2 = instance(F, Y)$$
$$O2 = Sol_{part}(head[2] - 1)$$
$$N_{\{Y1\}} = N_{\{Y2\}} = Sol_{part}(head[2] - S1)$$

Donc :

$$\boxed{Sol_{part} \equiv \lambda x.1}$$

Le temps d'exécution :

$$T_{part} \leq \tau + 1 \times T_{part}(head[2] - 1)$$
$$T_{part}(head[2]) = 1 + T_{part}(head[2] - 1)$$
$$T_{part}(0) = 1$$

$$\boxed{T_{part} \equiv \lambda x.x + 1}$$

3.4.3 Calcul de complexité pour le prédicat *qsort*

La taille des sorties en fonction des entrées

- Pour la première clause :
  $Sz^{(2)}_{qsort}(0) = 0.$
- Pour la deuxième clause :
  $Sz^{(2)}_{qsort}(head[1]) = body_4[3] = Sz^{(3)}_{append}(body_4[1], body_4[3])$

  or
  $Sz^{(3)}_{append} \equiv \lambda(x, y).x + y$

  d'où
  $$\begin{aligned} Sz^{(2)}_{qsort}(head[1]) &= body_4[1] + body_4[3] \\ &= Sz^{(2)}_{qsort}(body_2[1]) + 1 + Sz^{(2)}_{qsort}(body_3[1]) \\ &= Sz^{(2)}_{qsort}(Sz^{(3)}_{part}(head[1] - 1)) + 1 + \\ &\qquad Sz^{(2)}_{qsort}(Sz^{(4)}_{part}(head[1] - 1)) \end{aligned}$$

  or
  $Sz^{(3)}_{part} \equiv \lambda x.x$ et $Sz^{(4)}_{part} \equiv \lambda x.x$ d'où
  $Sz^{(2)}_{qsort}(head[1]) = 2 \times Sz^{(2)}_{qsort}(head[1] - 1) + 1$

  Donc :
  $$\boxed{Sz^{(2)}_{qsort} \equiv \lambda x.2^x - 1}$$

Précision :

Normalement, on aurait dû avoir la même taille en entrée et en sortie. Le fonctionnement de *qsort* est qu'il partitionne la liste en deux sous-listes en fonction du premier élément de la liste. Tous les éléments inférieurs au premier sont mis dans une des deux sous-listes et ceux supérieurs dans l'autre. On *relance récursivement qsort sur chacune des sous-listes*, et on concatène ces deux sous-listes.

Or pour la taille des deux sorties de *part*, Caslog va prendre la taille correspondant au pire des cas. On aura donc pour chaque sortie une taille égale à celle de l'entrée, alors que l'on aurait dû avoir la somme des deux tailles de sortie égale à celle de l'entrée. Ce qui nous aurait donné une équation de la forme :

$$Sz_{qsort}(x) = Sz_{qsort}(x - 1 - n) + Sz_{qsort}(n) + 1 \text{ avec } n \leq x - 1$$

avec $Sz_{qsort}(x)$ la taille de la sortie de *qsort* en fonction de celle de son entrée x,
$x - 1 - n$ étant le nombre d'éléments de la première sous-liste,
n le nombre d'éléments de la deuxième,
et $x - 1$ le nombre d'éléments en entrée de *part*.

De plus on a toujours $Sz_{qsort}(0) = 0$ avec la première clause de *qsort*. En résolvant ce système, on aurait bien eu $Sz_{qsort}(x) = x$.

Alors que le raisonnement de Caslog (dans le pire des cas) nous donne le système d'équations suivant :
$Sz_{qsort}(x) = Sz_{qsort}(x - 1) + Sz_{qsort}(x - 1) + 1$
$Sz_{qsort}(0) = 0.$
avec $x - 1$ le nombre d'éléments en entrée de *part*.
Et en résolvant ce système, Caslog obtient $Sz_{qsort}(x) = 2^x - 1$.

La taille de la relation :

qsort est récursif, donc :

$$\boxed{Rel_{qsort} = \infty}$$

Le nombre de solutions de qsort :

- Pour la première clause :
  $Sol_{qsort}(0) = 1.$
- Pour la deuxième clause :
  $First$ et $L1$ sont des arguments d'entrée, donc
  $N_{\{First\}} = N_{\{L1\}} = 1$
  et
  $I1 = min\{instance(first, L1), Rel_{part}\} = 1$
  $O1 = 1 \times Sol_{part}(head[1] - 1) = 1$
  $N_{\{Ls\}} = N_{\{Lg\}} = 1$
  $I2 = instance(Ls) = 1$
  $O2 = Sol_{qsort}(body_2[1]) = Sol_{qsort}(head[1] - 1)$
  $N_{\{Ls2\}} = Sol_{qsort}(head[1] - 1)$
  $I3 = instance(Lg) = 1$
  $O3 = Sol_{qsort}(body_3[1]) = Sol_{qsort}(head[1] - 1)$
  $N_{\{Lg2\}} = Sol_{qsort}(head[1] - 1)$
  $I4 = instance(Ls2) = Sol_{qsort}(head[1] - 1)$
  $O4 = Sol_{qsort}(head[1] - 1) \times Sol_{append}(body_4[1], body_4[2])$
  $O4 = Sol_{qsort}(head[1] - 1)$
  $N_{\{L2\}} = Sol_{qsort}(head[1] - 1)$

  $Sol_{qsort}(head[1]) = Sol_{qsort}(head[1] - 1)$
  donc :

$$\boxed{Sol_{qsort} \equiv \lambda x.1}$$

La complexité en temps de qsort :

On prend comme unité de complexité en temps le nombre de résolutions, on a $\tau = 1$

$T_{qsort}(head[1]) = 1 + T_{part}(head[1] - 1) +$
$Sol_{part}(head[1] - 1) \times T_{qsort}(Sz^{(3)}_{part}(head[1] - 1)) +$
$Sol_{part}(head[1] - 1) \times T_{qsort}(Sz^{(4)}_{part}(head[1] - 1)) +$
$Sol_{part}(head[1] - 1) \times Sol_{qsort}(Sz^{(3)}_{part}(head[1] - 1)) \times$
$Sol_{qsort}(Sz^{(4)}_{part}(head[1] - 1)) \times T_{append}(Sz^{(2)}_{qsort}(Sz^{(3)}_{part}(head[1] - 1)),$
$Szqsort^{(2)}(Sz^{(4)}_{part}(head[1] - 1) + 1)))$

d'où

$T_{qsort}(x) = 1 + x + 1 + 1 \times T_{qsort}(x - 1) + 1 \times Tqsort(x - 1) + 1 \times 1 \times 1 \times 2^{x-1}$

donc

$$\boxed{T_{qsort} \equiv \lambda x. \sum_{i=1}^{x}(i \times 2^{x-i}) + x \times 2^{x-1} + 2^{x+1} - 1}$$

Chapitre 4

Programmation de Caslog en ISO-Prolog

4.1 Le fonctionnement de Caslog

Caslog signifie *Complexity Analysis System for LOGic*, il s'agit d'un prototype d'implémentation des techniques d'analyses de prédicats vues précédemment.
Il est composé de cinq parties :

1. un préprocesseur avec plusieurs sous-modules :
   - analyseur des modes ;
   - analyseur de dépendance ;
   - analyseur des exclusions mutuelles entre les clauses ;
   - analyseur de type ;
   - analyseur de mesure (int, length, depth, void, size),
2. un module pour le calcul de taille des arguments,
3. un analyseur de nombre de solutions,
4. un analyseur de complexité en temps,
5. un résolveur d'équation différentielle.

Le mode, le type d'analyse et la mesure sont fournis via le programme.
L'analyseur de dépendance permet de construire le graphe de dépendance entre les arguments et les littéraux pour chaque clause.

L'analyseur d'exclusion mutuelle classifie les clauses dans des classes pour chaque prédicat.

Par exemple, pour :
$:-mode(list/1,[+]).$
$list([]).$
$list([_|Xs]):-list(Xs).$
on trouve [[1],[2]], ce qui signifie que les clauses 1 et 2 sont mutuellement exclusive c'est-à-dire à chaque noeud de l'arbre de dérivation, soit la clause 1 s'applique ou bien la clause 2, mais pas les deux.

La taille des arguments se calcule à partir des mesures données dans le programme. Et ainsi, on peut calculer la taille des arguments de sortie en fonction des tailles des arguments en entrée.

Le calcul du nombre de solutions d'un programme s'effectue à partir de deux composants :

- L'analyseur des tailles des relations :
  si le prédicat est une conjonction de contraintes binaires (sauf l'égalité) ou une conjonction et/ou disjonction de contraintes linéaires arithmétiques, on utilise un algorithme spécial pour calculer le nombre de relations. Sinon dans les autres cas, un algorithme général est utilisé (le même que celui utilisé pour calculer le nombre de solutions).
- L'analyseur du nombre de solution.

L'analyseur de complexité en temps combine les résultats des tailles des arguments et du nombre de solutions pour estimer une fonction de complexité en temps pour chaque prédicat.

Ainsi, pour des clauses récursives, les différents analyseurs fournissent des équations différentielles. Un résolveur d'équation différentielle est implémenté dans Caslog. Il peut résoudre plusieurs types d'équations différentielles :

- les équations différentielles linéaires de premier ordre ;
- les équations de second ordre ;
- les équations différentielles basées sur le paradigme "diviser pour régner" : de la forme $x(n) = x(n - i/b) + g(n)$;
- les équations différentielles dérivant des clauses avec comme mesure *size*.

4.2 Les différents modules de Caslog

Le module algebraic est composé de :

- **algebraic.pl** : pour manipuler les équations en formes normales,
- **diff_equ.pl** et **difference_equs.pl** : pour la résolution d'équations différentielles de type linéaire,
- **general_form.pl** : pour les transformations de forme normale en forme générale,
- **math.pl** : contient les deux fonctions mathématiques *log* et *pow* nécessaires pour la résolution d'équations,
- **maxmin.pl** : pour le calcul du maximum et du minimum de deux formes normales,
- **normal_form.pl** : toutes les procédures de transformation et de simplification, de forme normale en forme générale.

Le module color est composé de :

- **cspoly.pl** : pour estimer un polynôme chromatique d'un graphe donné,
- **disequality.pl** : pour les tests de différence pour déterminer si un prédicat est un GCP (Graph Chromatic Polynomial),
- **gcp.pl** : pour déterminer le nombre de solutions d'un GCP.

Le module csp est consacré à la manipulation de graph csp (Chromatic Polynomial Graph) :

- **csp.pl** : pour la manipulation de graphes consistants,
- **consistency.pl** : pour la construction de graphes consistants,

- **clique.pl** : contient des prédicats pour déterminer le nombre de n-cliques (avec n le nombre de variables), pour ainsi trouver le nombre de relations,
- **unfold.pl** : pour dérouler des prédicats de test,
- **constraint.pl** : contient les différents tests sur les contraintes des prédicats pour la construction du graphe CSP.

Le module dependency pour la construction et la manipulation de graphes de dépendance :

- **adg.pl** : pour les manipulations sur le graphe de dépendance des arguments dans chaque littéral $\longrightarrow$ adg(pos,pred,succ,mode)[1],
- **build_adg** : pour la construction du graphe de dépendance des arguments,
- **ldg.pl** : différentes procédures pour le graphe de dépendance sur les littéraux $\longrightarrow$ ldg(lit,edge)[2],
- **ldg_build.pl** : pour la construction du graphe de dépendance sur les littéraux,
- **dependency.pl** : pour la construction de graphe de dépendance à l'aide build_ldg et build_adg,
- **gvars.pl** : pour les variables closes, on a ainsi une liste de variables closes, $\longrightarrow$ gv(var,pos)[3],
- **position.pl** : pour définir la position des arguments ou des littéraux.

Le module determinacy est composé de :

- **determinacy.pl** : pour analyser si les prédicats sont déterministes c'est-à-dire pour toute entrée on aura toujours une seule et même solution,
- **mutual_exclusions** : pour calculer les différentes classes d'exclusion mutuelle de clauses.

Le module diff_equation est composé de :

- **diff_equ.pl** : pour résoudre les équations différentielles,
- **diff_equ_utility.pl** : test pour une équation donnée, s'il existe une solution à l'aide de la méthode à coefficient constant,
- **divide_conquer.pl** : pour résoudre les équations qui reposent sur le schéma "diviser pour régner", comme l'équation de *qsort*,
- **explicit_size.pl** : teste si une équation différentielle de base avec une mesure $term_size$ a une structure linéaire de premier ordre à coefficients égaux à 1 :
  $f(n) = f(arg(n,m)) + f(arg(n,m-1)) + ... + g(n)$,
  où $g(n)$ est soit d, $d \times n$ ou $arg(n,i)$,
- **first_order.pl** : pour manipuler les équations de premier ordre,
- **second_order.pl** : pour manipuler les équations de second ordre,
- **highter_order** : pour manipuler les équations d'ordre supérieur à 2,
- **implicit_size** : pour résoudre les équations de premier ordre de coefficient 1, de mesure $term_size$ contenant des éléments exprimés en fonction de l'arité ou de la taille d'un des arguments en entrée,

[1] pos : position de l'argument ; pred : prédécesseur ; succ : successeur ; mode : entrée ou sortie
[2] lit : littéral ; edge : arc qui associe la dépendance entre deux littéraux
[3] var : variable ; pos : sa position dans le littéral

- **list_size.pl** : teste si une équation différentielle de mesure *term_size* sur une liste est linéaire de premier ordre de coefficient 1 , c'est-à-dire :
  $f(n) = f(head(n)) + f(tail(n)) + g(n)$,
  où $g(n)$ est soit d, $d \times n$, $head(n)$, ou $tail(n)$,
- **mutual_size.pl** : pour résoudre les équations linéaires de premier ordre de coefficient 1 contenant des éléments particuliers comme Arg et Arity,
- **product.pl** : pour résoudre les équations non-linéaire exprimées sous forme de produit.

Le module init pour initialiser le système :

- **initsystem.pl** : pour construire la table des prédicats prédéfinis BT[4] et le graphe d'appel SCCG[5] et la liste des clauses,
- **buildin.pl** : pour la construction de la table BT de la forme :
  st(Pred/Arity, clause, mode, measure, mutex[6] , det[7] , size, solution, time, domain),
- **callgraph.pl** : pour la construction du graphe d'appel,
- **dec.pl** : pour manipuler les déclarations de type :- suivi de mode, measure ou domain,
- **scc.pl** : pour trouver les composants fortement connexes du graphe d'appel,
- **symtable.pl** : pour la construction de ST[8] pour les prédicats que l'on va définir de la forme :
  st(Pred/Arity, clause, mode, measure, mutex, det, size, solution, time).

Le module size pour calculer la taille des arguments d'entrées et de sorties des prédicats :

- **clause.pl** : ce fichier contient les prédicats nécessaires pour trouver toutes les informations au sujet des littéraux et des clauses,
- **ground_size.pl** : pour calculer la taille des arguments clos,
- **implicit_size.pl** : pour calculer la taille implicite des variables en entrée, dans la tête de la clause,
- **insert_size.pl** : pour insérer dans la table ST la liste des tailles des arguments des prédicats,
- **normalise.pl** : pour normaliser les équations des tailles, en exprimant la taille des sorties en fonction des tailles des entrées,
- **size_diff_equ.pl** : pour résoudre les équations récurrentes des tailles,
- **size.pl** : pour l'analyse des tailles des arguments pour les prédicats dans le programme,
- **term_diff.pl** : ce fichier contient des prédicats pour manipuler des fonctions *diff*, c'est-à-dire pour calculer des différences de taille entre deux termes,
- **term_size** : pour calculer la taille des termes dans un prédicat.

[4]BT : Buildin Table
[5]SCCG : Strongly Connected Components Graph
[6]mutex : Les classes mutuellement exclusives
[7]det : indique si un prédicat est déterministe
[8]ST : Symbol Table

Le module solution pour trouver le nombre de solutions en fonction des arguments d'entrées :

- **binding.pl** : pour calculer le *"pattern binding"* $< Rel_p, Sol_p >$
  Rel_p : la taille de la relation d'un prédicat p,
  Sol_p : le nombre de solutions d'un prédicat p,
- **comp_diff_equ.pl** : pour résoudre les équations différentielles complexes,
- **relation.pl** : pour chercher la taille des relations des prédicats dans le programme,
- **solution.pl** : ce fichier contient les procédures pour calculer le nombre de solutions dans le programme.

Le module time pour calculer le temps d'exécution :

- **time.pl** : contient les procédures pour calculer le temps d'exécution du programme.

Le module top est le module principal pour exécuter Caslog :

- **top.pl** : pour démarrer Caslog de la forme : *caslog(['file1.pl','file2.pl',...])*.,
- **ouput.pl** : concerne tous les affichages de réponse à l'écran,
- **error.pl** : pour gérer les erreurs,
- **utility.pl** : les procédures utilisées par Caslog comme addition, soustraction...

4.3 Les modifications apportées

Tout d'abord, Caslog avait été implémenté pour fonctionner sous une ancienne version de SICStus Prolog.

4.3.1 Les modifications pour faire fonctionner sous une version récente de SICStus

Les fichiers de Nai-Wei Lin étaient composés de sources en Prolog avec des prédicats non-ISO Prolog et un fichier math.c. Ce dernier était mis en place pour définir les fonctions mathématiques puissance et logarithme. On les a tout d'abord réécrites en Prolog dans algébraic/math.pl, pour ne plus avoir de fichier .c à importer.

Le **Makefile** permet de faire compiler tous les fichiers .pl par sicstus avec des commandes du type :
echo "prolog_flag(compiling,_,fastcode). fcompile('source/top/utility.pl')." | sicstus

Caslog.install permet de charger tous les fichiers prolog déjà compilés (.ql) et de sauvegarder l'état du système dans **caslog.init**. Puis **caslog.init** va être utilisé dans le lancement de **Caslog**, pour ne pas avoir à recompiler à chaque exécution.

L'exécutable Caslog pour SICStus Prolog.

Caslog utilise le shell sh par défaut. Si on démarre caslog sans aucun argument, caslog restore l'état du système à partir du fichier caslog.init et démarre *caslog([])* avec en paramètre une liste vide. Mais si en entrée on a plusieurs programmes, il restaure l'état du système, et lance Caslog sur chacun des programmes de la liste.

Dans le nouveau shell, on a rajouté d'autres lignes de codes pour pouvoir connecter le résolveur d'équations différentielles (PURRS). Maintenant, on a dans le programme un "setup" pour sicstus qui lance le makefile. Et on a renommé caslog.install par sicstus.loading.

4.3.2 Les modifications pour faire fonctionner sous GNU-Prolog

On a créé tout d'abord un "setup" *gprolog.install*, qui lance le makefile.gprolog. Comme le fcompile n'était pas conforme à la norme ISO-Prolog, on a utilisé le gplc (un compilateur) de GNU-Prolog pour compiler. Pour obtenir les fichiers .wbc :

gplc -w –no-singl-warn source/top/top.pl

On crée ainsi des fichiers wam : wam-for-byte-code.

Au démarrage de caslog, on charge les fichiers wam à partir du fichier gprolog.loading, au lieu de tout recompiler, et on lance caslog avec la liste des programmes que l'on veut analyser.

4.3.3 Mise en ISO-Prolog

Tout d'abord, pour manipuler des clauses contenant deux opérateurs prédéfinis qui se suivent, on a dû parcourir tout le code pour insérer des parenthèses.

Par exemple $p :- !, q$ est conforme à ISO-Prolog, mais pour manipuler cette clause, on doit rajouter des parenthèses autour de !, c'est-à-dire :

$p :- (!), q$

Ensuite, on a constaté que l'on ne pouvait pas redéfinir un prédicat qui existait déjà en prolog, on a donc dû en renommer certains.

Par exemple : *list* en *list_cas* dans utility.pl.

On obtenait des erreurs avec certains prédicats sous GNU-Prolog, on a donc dû les réécrire.

Par exemple, avant on avait le prédicat no_of_cuts suivant :

$no_of_cuts((Lit, Body), No) :-$
    $no_of_cuts(Body, Nos),$
    $(Lit == (!) ->$
        $No\ is\ Nos + 1;$
        $No = Nos).$
$no_of_cuts(Lit, No) :-$
    $nonsequence(Lit),$
    $(Lit == (!) ->$
        $No = 1;$
        $No = 0).$

Dans le cas où on avait une entrée de type (X, Y), ce prédicat effectuait deux exécutions, une avec $Lit = (X, Y)$ et une autre $(Lit, Body) = (X, Y)$. Alors que Caslog attendait une seule réponse pour continuer (la deuxième exécution).

On a donc réécrit le prédicat de la façon suivante :

```
no_of_cuts(L, No) :-
(L = (Lit, Body)->
        (no_of_cuts(Body, Nos),
        (Lit == (!)->
                No is Nos + 1;
                No = Nos));
        (L = Lit->
                (nonsequence(Lit),
                (Lit == (!)->
                        No = 1;
                        No = 0))
        )
).
```

On a mis en commentaire toutes les fonctions d'affichage pour le débuggage, qui n'étaient pas conforme à la norme ISO-Prolog.
Par exemple, on a commenté des sections de codes, comme celle-ci :

```
/* print_buildin_table(BT) :-
        tell(buildin_table),
        p_buildin_table(BT),
        told.

p_buildin_table(BT) :-
        var(BT).
p_buildin_table(BT) :-
        nonvar(BT),
        BT = [E|B],
        write(E),
        nl,
        p_buildin_table(B).*/
```

Mais il restait encore quelques prédicats non conformes à la norme ISO-Prolog, on a donc cherché des équivalents. Par exemple, on a remplacé *see/1* et *seen/0* (pour ouvrir et fermer un fichier) par *open/4* et *close/1* qui étaient leurs équivalents en ISO-Prolog. En revanche les prédicats donnant des informations sur le système (temps d'exécution) comme *statistics/2* n'ayant pas d'équivalent en ISO-Prolog, on les a laissés car ils fonctionnent au moins sous GNU-Prolog et SICStus Prolog.

Chapitre 5

Connexion de PURRS à Caslog

5.1 Présentation de PURRS

5.1.1 Les principales fonctionnalités

PURRS signifie Parma University's Recurrence Relation Solver. C'est un outil de résolution d'équations récurrentes développé en C++ par une équipe de chercheurs de l'Université de Parme. Parmi les membres de cette équipe, notre principal interlocuteur a été Roberto Bagnara ; les autres membres de l'équipe sont Tatiana Zolo, Alessandro Zaccagnini et Enea Zaffanella. PURRS peut résoudre ou donner une approximation sur des équations :

- récurrentes linéaires d'ordre fini à coefficients constants ;
- récurrentes linéaires de premier ordre à coefficients variables ;
- récurrentes généralisées i.e. les équations qui apparaissent dans l'analyse d'algorithme de type divisé pour régner ;
- non-linéaires d'ordre fini (quelques unes).

5.1.2 Format des équations d'entrées

Les équations données en entrée de PURRS doivent être de la forme :

- pour les équations d'ordre fini à coefficients constants :
  $x(n) = a_1 x(n-1) + ... + a_k x(n-k) + p(n)$
  a_i : des coefficients rationnels
  avec des conditions initiales de type : $x(i) = m$
  m et i étant des rationnels ;
- pour les équations de premier ordre à coefficients variables :
  $x(n) = a(n)x(n-1) + p(n)$
  avec ce type d'équation, PURRS essaie d'exprimer les solutions en utilisant des polynômes, des fonctions exponentielles et factorielles, ou une combinaison de ces fonctions ;
- pour les récurrences généralisées ("diviser pour régner") :
  $x(n) = ax(n/b) + p(n)$
  avec a positif, b supérieur à 1 et n/b est interprété comme une partie entière. Ce type d'équation est utilisé pour le calcul de complexités dans le pire des cas, comme pour les algorithmes de type merge-sort. Ici, les conditions initiales doivent commencer à 1.

5.1.3 Les symboles acceptés pour ces équations

Les entrées et les sorties contiennent les éléments suivants :

- $+, -, *, /, ^$(puissance), les parenthèses. Les constantes doivent être des entiers ou des fractions ;
- les signes de multiplication ne doivent pas être omis ;
- les lettres en minuscules sont considérées comme des paramètres ;
- les factorielles n peuvent être écrites en entrés n ! ou factorial(n) et en sortie, on a factorial(n) ;
- les symboles sum et prod écrits de la forme :
  $sum(symbol, initial_value, final_value, summand)$,
  $prod(symbol, initial_value, final_value, summand)$,
  avec *summand* qui est une fonction de *symbol*, $initial_value$ doit être un entier positif ou nul et $final_value$ peut dépendre de n (la variable de récurrence) ;
- les équations sont toujours réécrites en forme standard c'est-à-dire :
  si on a $2 \times x(n+1)$ alors PURRS va résoudre $x(n) = x(n-1)/2$;
- En sortie, on peut avoir des variables de la forme symbol$i$
  avec i un entier et symbol$i$ utilisé pour représenter de nouvelles variables qui doivent être uniques ;
- on peut aussi avoir $mod(n, k)$ qui représente le reste de la division de n par k.

5.2 Méthodes utilisées pour la connexion

5.2.1 Connexion effectuée par un shell

Pour connecter PURRS à Caslog, premièrement nous avons localisé dans Caslog, où les équations différentielles étaient construites, ensuite, on les a récupérées dans des fichiers différents :

- *sizefunction* : pour calculer la taille des arguments de sorties ;
- *solution* : pour le nombre de solutions ;
- *time* : pour la complexité en temps.Connexion

En utilisant un shell sh, on a transformé ces équations pour être conforme à PURRS. Quand Caslog a fini son exécution normale, on lance le shell qui va automatiquement afficher les résultats de PURRS, ce qui nous a demandé la réalisation de codes plus ou moins complexes en Shell :

```
cat equ1|sed s/`cat pred`/x/g>equ1
   cat equ1|sed s/`cat var`/n/g>equ1
   cat equ1|sed -e "s/exp(\([^,]*\),\([^)]*\))/(\1)^(\2)/g">equ1
   while [ !  -z `cat lvar` ]; do
   cat lvar|sed -e "s/\([^|]*\)\(|.*\)/\1/g">var0
   cat linit|sed -e "s/\([^|]*\)\(|.*\)/\1/g">init0
   if  !  cat var0|grep -s [$]>/dev/null ; then
if !  cat init0|grep -s [$]>/dev/null ; then
   echo " -I x(`cat var0`)=`cat init0`">>equ1
else
   echo "warning : purrs don't solve initial condtion  like :">>PurrsResult
   echo "x(0)=m with m a constant term without value">>PurrsResult
   echo "`cat pred`(`cat var0`)=`cat init0` ignored" >>PurrsResult
   echo "">>PurrsResult
fi
```

Ce code transformait une équation "equ1" ; il remplaçait le nom du prédicat (pred) par x, la variable de récurrence par n, les $exp(X, Y)$ par $X\hat{}Y$ et vérifiait si il existait des équations représentant des conditions initiales. Si les équations initiales ne permettaient pas la résolution par PURRS, on affichait un message d'erreur.

5.2.2 Connexion effectuée directement par Prolog

Suivant les conseils de Roberto Bagnara (Professeur à l'Université de Parme), nous avons par la suite décidé de mettre les équations au format de PURRS directement en Prolog sans passer par le shell ; ceci pour des raisons de simplicité et de clarté, alors on a créé un module ConnectPurrs contenant les fichier Prolog suivant :

- fomatToPurrs.pl : transforme les équations récupérées sous forme normale en une forme générale compatible avec PURRS,
- connectToPurrs.pl : écrit les équations transformées au format de PURRS dans des fichiers sous la forme : $rrs_driver\ -R\ x(n-1)+1\ -I\ x(0)+1$,
- utility.pl : contient des opérations standards nécessaires à la conversion des équations au format de PURRS.

On utilise toujours un shell qui nous sert uniquement d'interface pour lancer PURRS avec les équations obtenues précédemment.

5.2.3 Exemple de problèmes rencontrés et résolus

Les nombres à virgule :

Un de nos problèmes principaux était que PURRS n'accepte pas les nombres à virgule, alors on a créé un prédicat qui les transforment en fraction :

```
floatTofrac(Fl,Fr):-
A is Fl*1.0e+4,
Ac is ceiling(A),
Bc is ceiling(1.0e+4),
pgcd(Ac,Bc,C),
Flp is Ac/C,D is ceiling(Flp),
Frp is Bc/C,E is ceiling(Frp),
Fr=D/E.
```

On multiplie le nombre par 10^4 (ce qui nous donnera une précision à 10^{-4} près), puis on prend la partie entière, et on recherche le pgcd entre ce nombre et 10000. Ensuite on divise ces deux nombres par leur pgcd pour obtenir une fraction réduite. Par exemple :

$5.7425 \times 10^4 = 57425$
$pgcd(57425, 10000) = 25$
$57425/25 = 2297$
$10000/25 = 400$

On remplace donc 5.745 par la fraction $\frac{2297}{400}$

Les fonctions arity, arg, head, tail :

L'exécution du prédicat $flatten$ par Caslog renvoie le système d'équations différentielles suivant :

$flatten(tail(\$(0,1))) + flatten(head(\$(0,1)))$
$flatten(1) = 1$
$flatten(0) = 0$

avec $tail(\$(0,1))$ qui représente la taille du reste de la liste en entrée, $head(\$(0,1))$ qui représente la taille de la tête de la liste, et la variable de récurrence étant $\$(0,1)$.

$flatten$ est de mesure $list_length$ alors d'après la définition de la taille d'un terme pour les $list_length$ du chapitre 2, on peut majorer head et tail par :

$head(\$(0,1)) \leq \$(0,1) - 1$

et $tail(\$(0,1)) \leq \$(0,1) - 1$

pour $\$(0,1) > 1$.

Ensuite avec le prédicat $flat$, on a les équations différentielles suivantes :

$sum(\$i, 1, arity(\$(0,1)), flat(arg(\$(0,1), \$i))) + 1$

$flat(1) = 1$

arg utilise des arguments de mesure $term_depth$.

$arg(\$(0,1), \$i)$ représente la taille du $i^{ème}$ argument de $\$(0,1)$.

D'après la définition de la taille d'un terme de mesure $term_deph$ au chapitre 2, alors, on a :

$arg(_, \$(0,1)) \leq \$(0,1) - 1$

$arity(\$(0,1))$ où $\$(0,1)$ est de mesure $term_deph$:

D'après la définition de la taille d'un terme au chapitre 2,

on a :

$arity(\$(0,1)) \leq \$(0,1) - 1$

Les équations du type "diviser pour régner" :

PURRS n'accepte que les récurrences du type $x(n) = x(n/k)$ ou $x(n) = x(n-k)$ avec k un entier. Dans le cas où $x(n) = x(n/k + c)$ avec k un entier et c un entier ou un terme en fonction de n, alors on garde que n/k dans la récurrence, ce qui nous donne $x(n) = x(n/k)$ que l'on met à l'entrée de PURRS.

Cas de prédicats dépendants entre eux :

Pour le prédicat $flat3$, on a des équations du type :

$flat3(n) = flat3(n-1) + flat(2, 1, 1/2 \times n)$

$flat3(0) = 1$

avec $flat(2, 1, 1/2 \times n)$ correspondant à la taille de la deuxième variable de la première clause définissant flat avec $1/2 \times n$ en entrée.

Pour résoudre ce problème, on a stocké ces équations dans une liste avec toutes les informations nécessaires sur le prédicat, puis on a laissé Caslog résoudre l'équation associée à $flat$ et on a substitué la valeur adéquate dans l'équation de $flat3$.

Problème avec msort

La taille des entrées en fonction des sorties

Tout d'abord, en utilisant les définitions sur les tailles des termes vues au chapitre 2 et le graphe de dépendance, on a le déroulement suivant :

– **divide**

$:-mode(divide/3,[+,-,-]).$
$:-measure(divide/3,[length,length,length]).$
$divide([],[],[]).$
$divide([X],[X],[]).$
$divide([X1,X2\|L],[X1\|L1],[X2\|L2]):-$
$divide(L,L1,L2).$

$Sz^2_{divide}(head[1]) = 1 + Sz^2_{divide}(head[1]-2)$
d'après les deux premières clauses, on a :
$Sz^2_{divide}(0) = 0$
$Sz^2_{divide}(1) = 1$
d'où :
$Sz^2_{divide}(head[1]) = \frac{head[1]}{2}$ si $head[1]$ est pair,
$Sz^2_{divide}(head[1]) = \frac{head[1]}{2} + 1$ si $head[1]$ est impair

d'où :

$$\boxed{Sz^2_{divide}(head[1]) \leq \frac{head[1]}{2} + 1}$$

de même on a :
$Sz^2_{divide}(head[1]) = 1 + Sz^2_{divide}(head[1]-2)$
d'après les deux premières clauses, on a :
$Sz^2_{divide}(0) = 0$
$Sz^2_{divide}(1) = 0$
d'où :

$$\boxed{Sz^2_{divide}(head[1]) \leq \frac{head[1]}{2}}$$

– **merge**

$: -mode(merge/3, [+, +, -]).$
$: -measure(merge/3, [length, length, length]).$

$merge(L, [], L).$
$merge([], L, L) : -$
$\quad L \setminus == [].$
$merge([X1|L1], [X2|L2], [X1|L]) : -$
$\quad X1 >= X2,$
$\quad merge(L1, [X2|L2], L).$
$merge([X1|L1], [X2|L2], [X2|L]) : -$
$\quad X1 < X2,$
$\quad merge([X1|L1], L2, L).$

$Sz^3_{merge}(head[1], 0) = head[1]$
$Sz^3_{merge}(0, head[2]) = head[2]$
$Sz^3_{merge}(head[1], head[2]) = 1 + Sz^3_{merge}(head[1] - 1, head[2])$
$Sz^3_{merge}(head[1], head[2]) = 1 + Sz^3_{merge}(head[1], head[2] - 1)$
d'où :

$$\boxed{Sz^3_{merge}(head[1], head[2]) = head[1] + head[2]}$$

– **msort**

$: -mode(msort/2, [+, -]).$
$: -measure(msort/2, [length, length]).$

$msort([], []).$
$msort([X], [X]).$
$msort([X1, X2|L], R) : -$
$\quad divide([X1, X2|L], L1, L2),$
$\quad msort(L1, R1),$
$\quad msort(L2, R2),$
$\quad merge(R1, R2, R).$

$Sz^2_{msort}(0) = 0$
$Sz^2_{msort}(1) = 1$
$Sz^2_{msort}(head[1]) = body_4[3]$
$= Sz^2_{msort}(body_2[1]) + Sz^2_{msort}(body_3[1])$
$= Sz_m sort^2(\frac{head[1]}{2} + 1) + z_m sort^2(\frac{head[1]}{2})$
on a système d'équation de la forme :
$f(0) = 0$
$f(1) = 1$
$f(x) = f(\frac{x}{2} + 1) + f(\frac{x}{2})$
Le système précédent est incohérent, avec la méthode de Caslog, on ne peut pas arriver à un résultat satisfaisant.

Lors de l'exécution de Caslog, on a en sortie de divide/3 les tailles suivantes : pour la première sortie : $-1/4 \times (-1)\hat{}n \ + \ n/2 \ + \ 1/4$
et la deuxième : $1/4 \times (-1)\hat{}n \ + \ n/2 \ - \ 1/4$

Ces équations indiquent que dans le cas d'une entrée comportant un nombre impair d'éléments, on aura $\lceil n/2 \rceil$ éléments dans la première sortie et $\lfloor n/2 \rfloor$ dans la deuxième. Comme *msort* dépend de *divide*, donc la taille de la sortie de *msort* va dépendre de ces résultats. On aura donc une équation du type :

$x(n) = x(-1/4 \times (-1)\hat{}n \ + \ n/2 \ + \ 1/4) + x(1/4 \times (-1)\hat{}n \ + \ n/2 \ - \ 1/4)$, qui ne correspond à aucun type d'équation connu par Caslog et PURRS.

Une solution serait d'inclure dans le vocabulaire de Caslog les parties par valeur inférieure et supérieure d'une expression.

5.3 Différence entre Caslog et PURRS

La principale différence entre les résultats de PURRS et Caslog est que PURRS renvoie au moins les mêmes résultats que Caslog ou plus simplifiés. Ceci nous permet d'avoir un résultat plus clair et plus précis avec PURRS.

Calcul des sommes

PURRS calcule la somme de termes contrairement à Caslog, par exemple :
pour la complexité en temps de *qsort*,
Caslog retourne :

$[sum(\$j, 1, \$1, exp(2, \$1 - \$j) \times \$j) + exp(2, \$1 - 1) \times \$1 + 2.0 \times exp(2, \$1) - 1.0]$

alors que PURRS retourne :

$qsort(n) \ =< \ -3 + 4 \times 2\hat{}n + 1/2 \times 2\hat{}n \times n - n.$

Simplification des résultats

PURRS simplifie les résultats contrairement à Caslog, par exemple pour la complexité en temps de *generator*,
Caslog retourne :
$[-(exp(exp(\$2, -1) - 1, -1) \times exp(\$2, \$1 - 1)) - 2 \times (exp(exp(\$2, -1) - 1, -1) \times exp(\$2, \$1)) + exp(exp(\$2, -1) - 1, -1) \times exp(\$2, -1) + 2 \times exp(exp(\$2, -1) - 1, -1) + exp(\$2, \$1)]$

alors que PURRS retourne :

$generator(n) =< -2 \times p\hat{}(1 + n) - (-1 + p)\hat{}(-1) - 2 \times p \times (-1 + p)\hat{}(-1) + p\hat{}(1 + n) \times (-1 + p)\hat{}(-1) + 2 \times p\hat{}(2 + n) \times (-1 + p)\hat{}(-1).$
avec $p \ = \ \$(0, 2)$.

Chapitre 6

Méthode de travail

Durant la première période de notre TER, nous avons travaillé selon le principe d'une réunion hebdomadaire avec Frédéric Mesnard et Etienne Payet, ce qui nous a permis de renforcer nos bases théoriques dans l'étude des complexités en programmation logique. Ainsi, nous avons eu une bonne gestion de notre temps de travail et on a pu bien avancer dans notre partie théorique. A la fin de cette période, on a rendu un pré-rapport (début Janvier 2003).

On a décidé dès le début de le rédiger en LaTeX, d'une part car il devait contenir de nombreuses formules mathématiques et d'autre part, il devrait être lu par d'autres Scientifiques (LaTeX étant le langage de référence pour les documents scientifiques).

Ensuite nous nous sommes consacrés à la réécriture du logiciel Caslog pour le rendre conforme à la norme ISO-Prolog, Caslog étant programmé initialement pour fonctionner sous SICStus-Prolog.

Durant cette deuxième période, Frédéric Mesnard étant en Australie, nous avons changé de méthode de travail et décidé d'envoyer de manière régulière l'avancement de nos travaux (effectués principalement sous Linux Mandrake 9.1) par mail à Frédéric Mesnard.

De plus, pour la connexion de PURRS à Caslog, nous avons travaillé en collaboration avec une équipe de l'université de Parme dirigée par Roberto Bagnara. On a commencé à correspondre avec cette équipe par l'intermédiaire de Frédéric Mesnard. Par la suite, pour des raisons de commodités, nous avons décidé d'un commun accord de dialoguer directement avec l'équipe de Roberto Bagnara. Cette communication s'effectuait toujours par mail et entièrement en anglais.

Chapitre 7

Conclusion

Notre TER nous a permis de manipuler des concepts de complexité dans la programmation logique. Le fait de travailler sur de la documentation scientifique en anglais nous avait paru comme une difficulté au début, mais s'est avéré par la suite très bénéfique. En effet, cela nous a permis de nous familiariser avec le vocabulaire scientifique en anglais, d'autant plus que la quasi-totalité des documents portant sur la programmation logique et l'estimation de complexités sont rédigés en anglais.

Comme nous l'avons remarqué dans le section 5.2 avec le prédicat *msort*, l'insertion dans le vocabulaire de Caslog des parties par valeur inférieure et supérieure d'une expression pourrait être l'objet d'un travail futur.

Par la réalisation de ce TER, on a pu approfondir ces concepts de complexités en détail et on a eu un léger aperçu de ce que pouvait apporter le travail de plusieurs équipes sur la réalisation d'un même projet. Ceci devrait nous être très utile dans la suite nos études ainsi que dans notre vie professionelle.

Bibliographie

[1] Saumya K. Debray, Nai-Wei Lin. *Cost Analysis of Logic Programs.* Departement of Computer Science. The University of Arizona. Tucson, AZ 85721. August 13, 1992.

[2] Saumya K. Debray and Nai-Wei Lin. *Automatic Complexity Analysis of Logic Programs.* Departement of computer Science. The Univertsity of Arizona. Tucson, AZ 85721, USA.

[3] S. K. Debray (The University of Arizona), P. López García et M. Hermenegildo (Universidad Politecnica de Madrid), N.-W. Lin (National Chung Cheng University - Taïwan). *Estimating the Computational Cost of Logic Programs.*

[4] Michael A. Covington. *ISO Prolog : A summary of the draft Proposed Standard (Revised)* Micheal A. Covington, Artificial Intelligence Center, The University of Georgia Athens, Georgia 30602-7415 USA August 1, 1994.

[5] Michael A. Covington. *Some Coding Guidelines for Prolog* Artificial Intelligence Center, The University of Georgia Athens, Georgia 30602-7415 USA 23 August 2001.

[6] Frédéric Mesnard. *Cours de Prolog de Licence Informatique* Université de la Réunion.

[7] Tobias Oetiker, Hubert Partl, Irene Hyna et Elisabeth Schlegl traduit par Matthieu Herrb. *Une courte (?) introduction à LaTeX* 2_ϵ